Optimizing HVAC System for Energy Efficiency

Introduction

This case explores the challenge of optimizing the indoor temperature of the Mudd Engineering Building at Columbia University. The objective is to minimize energy consumption while ensuring a comfortable environment for students and faculty.

Background

The Mudd Engineering Building, a hub for engineering activities at Columbia University, requires a well-regulated HVAC system to maintain a conducive learning and research environment.

Problem Statement

Our task is to determine the optimal indoor temperature settings for the building over a 24-hour period. The goal is to minimize energy usage while keeping the indoor conditions within a comfortable range.

Model and Assumptions

- We aim to maintain an ideal indoor temperature for the building.
- The energy consumption is influenced by the difference between the desired indoor temperature and the fluctuating outdoor temperature.
- A constant rate of energy use is assumed per degree of temperature difference (assume this rate to be 0.5 for ease).
- Indoor temperature fluctuations should be within a range for comfort.

Optimization Framework

The task is to optimize the HVAC settings to reduce energy consumption while adhering to indoor comfort standards.

Energy Consumption Function

For ease of modeling the energy consumption function is defined as:

$$E = \sum_{i=1}^{24} rate \times |T_{indoor}(i) - T_{outdoor}(i)| \tag{1}$$

Here, $T_{indoor}(i)$ and $T_{outdoor}(i)$ represent the indoor and outdoor temperatures at hour i, respectively.

Optimizing HVAC System for Energy Efficiency

Implementation
We will use recorded outdoor temperature data for a typical day and apply an optimization algorithm to determine the most energy-efficient indoor temperature settings. The hourly outdoor temperature data is presented in the table 1.

Constraints

- The indoor temperature must remain within a comfortable range, not deviating more than 3°C from the desired set point of 22 degrees.
- To reflect the thermal inertia of the building, the indoor temperature cannot change too rapidly from one hour to the next.
- Energy efficiency is a priority, but not at the cost of occupant comfort and safety.

Results and Analysis The optimization process will yield a schedule of hourly temperature settings.

Hour of Day	Outdoor Temperature (°C)
0	10
1	9
2	8
3	7
4	7
5	6
6	7
7	9
8	12
9	15
10	18
11	20
12	22
13	23
14	24
15	24
16	23
17	22
18	20
19	18
20	16
21	14
22	12
23	11

Table 1: Hourly outdoor temperature for November 32nd 2025

Optimizing HVAC System for Energy Efficiency

Correct Answer 1.

Objective Function

The objective is to minimize the total energy consumption of the HVAC system over a 24-hour period, which is defined as:

$$\text{Minimize } E = \sum_{i=0}^{23} rate \times |T_{indoor}(i) - T_{outdoor}(i)| \tag{2}$$

where:

- E *is the total energy consumption.*
- $\boldsymbol{rate}$ *is the energy consumption rate per degree difference between indoor and outdoor temperatures.*
- $T_{indoor}(i)$ *is the indoor temperature at the* i*-th hour (decision variables).*
- $T_{outdoor}(i)$ *is the outdoor temperature at the* i*-th hour.*

Constraints

The optimization is subject to the following constraints:

Desired Temperature Range:

$$22\,^{\circ}C - 2\,^{\circ}C \leq T_{indoor}(i) \leq 22\,^{\circ}C + 2\,^{\circ}C, \quad \forall i \in \{0, 1, \ldots, 23\} \tag{3}$$

This constraint ensures that the indoor temperature remains within a comfortable range, not deviating more than 3°C from the desired set point of 22°C.

Thermal Inertia Constraint:

Let Δ *denote the maximum temperature change per hour (for example,* $\Delta = 1^{\circ}C$*). Then for every pair of consecutive hours:*

$$|T_{indoor}(i+1) - T_{indoor}(i)| \leq \Delta, \qquad \forall i \in \{0, 1, \ldots, 22\}.$$

This constraint ensures that the indoor temperature schedule evolves gradually over time and prevents unrealistic jumps in temperature settings.

Optimizing HVAC System for Energy Efficiency

```
import numpy as np
from scipy.optimize import minimize

# Sample hourly outdoor temperatures (24 hours)
outdoor_temps = [10, 9, 8, 7, 7, 6, 7, 9, 12, 15, 18, 20, 22, 23, 24, 24,
    23, 22, 20, 18, 16, 14, 12, 11]

# Desired indoor temperature
desired_temp = 22  # in degrees Celsius
temp_range = 3  # permissible range from the desired temperature
temp_inertia = 1 # thermal inertia parameter (max hourly temperature change
    )
rate = 0.5  # hypothetical energy rate

# Energy consumption function
def energy_consumption(indoor_temps, outdoor_temps, rate):
    return sum(rate * abs(indoor - outdoor)
               for indoor, outdoor in zip(indoor_temps, outdoor_temps))

def objective(indoor_temps):
    return energy_consumption(indoor_temps, outdoor_temps, rate)

# Constraints

# Comfort range constraints
def constraint_upper(indoor_temps):
    return (desired_temp + temp_range) - indoor_temps

def constraint_lower(indoor_temps):
    return indoor_temps - (desired_temp - temp_range)

# Thermal inertia constraints
# |T(i+1) - T(i)| <= temp_inertia

# Implemented as:
#   T(i+1) - T(i) <= temp_inertia
#   -(T(i+1) - T(i)) <= temp_inertia
def constraint_rate_up(indoor_temps):
    return np.array([temp_inertia - (indoor_temps[i+1] - indoor_temps[i])
                     for i in range(23)])

def constraint_rate_down(indoor_temps):
    return np.array([temp_inertia + (indoor_temps[i+1] - indoor_temps[i])
                     for i in range(23)])


# Optimization setup

# Initial guess (keeping indoor temperature constant at desired_temp)
initial_guess = [desired_temp] * 24

# Define the bounds for each hour's temperature
bounds = [(desired_temp - temp_range, desired_temp + temp_range)] * 24
```

Optimizing HVAC System for Energy Efficiency

```
cons = [
    {'type': 'ineq', 'fun': constraint_upper},
    {'type': 'ineq', 'fun': constraint_lower},
    {'type': 'ineq', 'fun': constraint_rate_up},
    {'type': 'ineq', 'fun': constraint_rate_down}
]

result = minimize(objective,
                  initial_guess,
                  method='SLSQP',
                  bounds=bounds,
                  constraints=cons)
# Results
if result.success:
    optimized_temps = result.x
    print("Optimized indoor temperatures:", optimized_temps)
    print("Total energy consumption:", objective(optimized_temps))
else:
    print("Optimization failed:", result.message)
```